Search Route in Flask

In a Flask web application, a Search Route allows users to search for specific data (like names, emails, products, notes, etc.) from a database.

Let's understand this in 5 key parts:

🧩 1. Handling POST Request and Input Data

➤ Concept:

When you create a search form in HTML and submit it, Flask needs to receive that search input from the user.

The HTML form can send data using two methods:

- GET → data visible in the URL (like Google search)
- POST → data hidden from the URL (used when data is sensitive or long)

In Flask, you handle both with:

from flask import request

➤ Example:

```
@app.route('/search', methods=['GET', 'POST'])
def search():
    if request.method == 'POST':
        keyword = request.form['keyword']   # fetch the user input from form
        print(keyword)  # for debugging
        return f"You searched for: {keyword}"
    return render_template('search.html')
```

➤ HTML form:

```
<form method="POST" action="/search">
    <input type="text" name="keyword" placeholder="Enter search term">
    <button type="submit">Search</button>
</form>
```

🧠 Understanding:

- When user types something and clicks "Search", the browser sends that text to Flask.
- Flask reads it using request.form['keyword'].

🧠 2. Input Validation Using Regex

➤ Concept:

You should always validate user input before using it — to prevent:

- Errors
- Wrong queries
- SQL Injection attacks

For simple validation (like only allowing alphabets or digits), we use Regular Expressions (Regex).

➤ Example:

```
import re


@app.route('/search', methods=['POST'])

def search():

  keyword = request.form['keyword']


  # Regex: Allow only letters, numbers, and spaces

  if not re.match("^[A-Za-z0-9 ]+$", keyword):

    return "❌ Invalid input! Only letters and numbers are allowed."


  return f"✅ Valid keyword: {keyword}"
```

🧠 Meaning:

- re.match("^[A-Za-z0-9]+$", keyword) → checks that the keyword contains only alphabets, digits, and spaces.
- If not valid → show an error message.

🚨 3. Invalid Input Handling

When user enters something invalid (like @#$%), we shouldn't crash the app.
We should handle it gracefully.

You can:

- Show a message on the same page

- Redirect to the search page again with a flash message

➤ Example with Flash:

```
from flask import flash, redirect, url_for


@app.route('/search', methods=['POST'])
def search():
    keyword = request.form['keyword']
    if not re.match("^[A-Za-z0-9 ]+$", keyword):
        flash("Invalid input! Please use only letters and numbers.")
        return redirect(url_for('search_page'))
```

➤ HTML Template (search.html):

```
{% with messages = get_flashed_messages() %}
  {% if messages %}
    {% for msg in messages %}
      <p style="color:red;">{{ msg }}</p>
    {% endfor %}
  {% endif %}
{% endwith %}


<form method="POST" action="/search">
    <input type="text" name="keyword" placeholder="Enter search term">
    <button type="submit">Search</button>
</form>
```

✅ Output:
If user types abc@, it shows:
"Invalid input! Please use only letters and numbers."

🗄️ 4. Query Execution using SQL LIKE operator

Once input is valid, we can search for similar records in our database.

➤ Concept:

LIKE operator in SQL is used for pattern matching.

Example SQL query:

SELECT * FROM students WHERE name LIKE '%john%';

It means:
→ Find all names that contain “john” (like John, Johnson, Johnny, etc.)

➤ Flask Example:

```
import sqlite3


@app.route('/search', methods=['POST'])
def search():
    keyword = request.form['keyword']


    if not re.match("^[A-Za-z0-9 ]+$", keyword):
        return "Invalid input!"


    # connect to database
    conn = sqlite3.connect('students.db')
    cur = conn.cursor()


    # Use parameterized query to avoid SQL Injection
    query = "SELECT * FROM students WHERE name LIKE ?"
    cur.execute(query, ('%' + keyword + '%',))
    results = cur.fetchall()
    conn.close()


    return render_template('results.html', data=results)
```

🧠 Understanding:

- '%' + keyword + '%' adds wildcards around the keyword → matches anywhere in the string.
- cur.fetchall() → returns all matching rows.

- Always use parameterized queries (with ?) to avoid SQL injection.

🖥️ 5. Displaying Search Results

Now, you need to display the matching records on the web page.

➤ Example (results.html):

```
<h2>Search Results</h2>
{% if data %}
 <table border="1" cellpadding="5">
  <tr>
   <th>ID</th>
   <th>Name</th>
   <th>Course</th>
  </tr>
  {% for row in data %}
  <tr>
   <td>{{ row[0] }}</td>
   <td>{{ row[1] }}</td>
   <td>{{ row[2] }}</td>
  </tr>
  {% endfor %}
 </table>
{% else %}
 <p>No results found.</p>
{% endif %}
```

✅ Output Example:

Search: John

Results:

ID	Name	Course

| 1 | John | Python |

| 3 | Johnny | Java |

⚙️ Full Working Example (Mini Project Level)

Here's a simple complete Flask search route setup 👇

app.py

```
from flask import Flask, render_template, request
import sqlite3, re

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('search.html')

@app.route('/search', methods=['POST'])
def search():
    keyword = request.form['keyword']

    # Input validation
    if not re.match("^[A-Za-z0-9 ]+$", keyword):
        return "❌ Invalid input! Use only letters and numbers."

    # DB Connection
    conn = sqlite3.connect('students.db')
    cur = conn.cursor()
    query = "SELECT * FROM students WHERE name LIKE ?"
    cur.execute(query, ('%' + keyword + '%',))
    results = cur.fetchall()
    conn.close()
```

```
    return render_template('results.html', data=results, keyword=keyword)

if __name__ == '__main__':
    app.run(debug=True)
```

search.html

```
<h2>Student Search</h2>
<form method="POST" action="/search">
    <input type="text" name="keyword" placeholder="Enter student name">
    <button type="submit">Search</button>
</form>
```

results.html

```
<h2>Results for "{{ keyword }}"</h2>
{% if data %}
  <table border="1">
    <tr><th>ID</th><th>Name</th><th>Course</th></tr>
    {% for row in data %}
      <tr>
        <td>{{ row[0] }}</td>
        <td>{{ row[1] }}</td>
        <td>{{ row[2] }}</td>
      </tr>
    {% endfor %}
  </table>
{% else %}
  <p>No records found.</p>
{% endif %}
```

🗒️ Summary (Easy to Remember)

Step	Concept	Flask Feature Used
1	Handle input from form	request.form
2	Validate input	re.match()
3	Handle invalid data	Flash / Redirect
4	Execute DB query	sqlite3, SQL LIKE
5	Show results	render_template()

💡 Project Tips

✅ Always validate user input before sending to DB.
✅ Use parameterized queries (?) — never string concatenation in SQL.
✅ Keep a friendly message for “No results found”.
✅ Make search results page neat with a table or card layout.
✅ Optional: Add pagination for long lists.

🔍 **Mini Project: Student Search System (Flask + MySQL)**

🎯 **Project Objective**

A small Flask web app where:

- User can **search student details by name**
- Input is **validated** using regex
- Records are fetched from a **MySQL database** using LIKE
- Results are displayed neatly on a results page

🧱 **Project Structure**

student_search/

│

├── app.py

├── templates/

│ ├── search.html

│ └── results.html

├── static/

│ └── style.css (optional for design)

⚙️ **Step 1: Setup MySQL Database**

Before coding Flask, create a simple table in MySQL.

👉 **Open MySQL and Run:**

```
CREATE DATABASE studentdb;


USE studentdb;


CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    course VARCHAR(100)
);
```

```
INSERT INTO students (name, course) VALUES
('John', 'Python'),
('Johnny', 'Java'),
('Alice', 'C++'),
('Robert', 'HTML'),
('Bob', 'Python');
```

✅ This will create a sample database with 5 student records.

🧩 Step 2: Install Required Python Packages

In your terminal or command prompt, install:

```
pip install flask mysql-connector-python
```

🧠 Step 3: Flask App Code (app.py)

```
from flask import Flask, render_template, request
import mysql.connector
import re

app = Flask(__name__)

# Database connection function
def get_db_connection():
    conn = mysql.connector.connect(
        host="localhost",
        user="root",        # replace with your MySQL username
        password="root",    # replace with your MySQL password
        database="studentdb"
    )
    return conn
```

```
@app.route('/')
def home():
    return render_template('search.html')



@app.route('/search', methods=['POST'])
def search():
    keyword = request.form['keyword']

    # 1️⃣ Validate input
    if not re.match("^[A-Za-z0-9 ]+$", keyword):
        return "❌ Invalid input! Only letters and numbers are allowed."

    # 2️⃣ Connect to DB
    conn = get_db_connection()
    cur = conn.cursor()

    # 3️⃣ Execute query using LIKE
    query = "SELECT * FROM students WHERE name LIKE %s"
    cur.execute(query, ('%' + keyword + '%',))
    results = cur.fetchall()

    # 4️⃣ Close DB
    conn.close()

    # 5️⃣ Send results to HTML
```

```
    return render_template('results.html', data=results, keyword=keyword)


if __name__ == '__main__':
    app.run(debug=True)
```

🖼️ **Step 4: Frontend Files**

templates/search.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Search</title>
    <style>
        body {
            font-family: Arial;
            background: #f2f2f2;
            text-align: center;
            padding-top: 100px;
        }
        form {
            background: white;
            display: inline-block;
            padding: 20px;
            border-radius: 10px;
            box-shadow: 0px 0px 10px gray;
        }
        input, button {
            padding: 10px;
```

```
            margin: 5px;
        }
        button {
            background: blue;
            color: white;
            border: none;
            cursor: pointer;
        }
        button:hover {
            background: darkblue;
        }
    </style>
</head>
<body>
    <h2>🔍 Student Search System</h2>
    <form method="POST" action="/search">
        <input type="text" name="keyword" placeholder="Enter student name" required>
        <button type="submit">Search</button>
    </form>
</body>
</html>
```

---

templates/results.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Search Results</title>
    <style>
```

```
        body {
            font-family: Arial;
            background: #e8f0fe;
            text-align: center;
            padding-top: 50px;
        }
        table {
            margin: auto;
            border-collapse: collapse;
            width: 60%;
            background: white;
            box-shadow: 0px 0px 10px gray;
        }
        th, td {
            padding: 10px;
            border: 1px solid #ddd;
        }
        th {
            background-color: #007bff;
            color: white;
        }
        a {
            text-decoration: none;
            color: blue;
        }
    </style>
</head>
<body>
```

```
    <h2>Results for "{{ keyword }}"</h2>

    {% if data %}
      <table>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Course</th>
        </tr>
        {% for row in data %}
        <tr>
          <td>{{ row[0] }}</td>
          <td>{{ row[1] }}</td>
          <td>{{ row[2] }}</td>
        </tr>.
        {% endfor %}
      </table>
    {% else %}
      <p>No records found.</p>
    {% endif %}

    <br>
    <a href="/">🔙 Go Back</a>
</body>
</html>
```

🧠 **Step 5: How It Works (Step-by-Step Flow)**

Step	What Happens	Flask Concept Used
1️⃣	User opens / → search page loads	render_template()
2️⃣	User enters a name and clicks **Search**	HTML Form with POST
3️⃣	Flask reads input via request.form['keyword']	request
4️⃣	Input validated using Regex	re.match()
5️⃣	Query executed using LIKE '%keyword%'	MySQL + LIKE
6️⃣	Data returned and displayed in table	render_template() with Jinja

---

✅ Example Outputs

1️⃣ Input: John

Results for "John"

ID	Name	Course
1	John	Python
2	Johnny	Java

2️⃣ Input: @abc

❌ Invalid input! Only letters and numbers are allowed.

3️⃣ Input: Bob

Results for "Bob"

ID	Name	Course
5	Bob	Python

🗒️ Summary

Step	Concept	Flask Feature
1	Handle form data	request.form
2	Validate input	re.match()

Step	Concept	Flask Feature
3	Connect MySQL	mysql.connector
4	Query with LIKE	SQL query with %
5	Display results	Jinja templating

🌟 **Advantages of Teaching This Mini Project**

✅ Beginner-friendly and explains **core Flask flow**
✅ Real MySQL database integration
✅ Regex validation concept for safe input
✅ Simple UI with meaningful output
✅ Perfect as a **lab exercise or assignment**

MySQL Notes: = vs LIKE Operator (searching purpose we can use like or = operators)

🔹 **1. = (Equal Operator)**

✔ **What it does**

- Used to check **exact match**
- Value must be **exactly same**

✔ **Syntax**

```
SELECT * FROM student WHERE sname = 'raji';
```

✔ **Example**

sname in table	Search value	Result
raji	raji	✅ Match
Raji	raji	❌ No match (depends on collation)
raj	raji	❌ No match

✔ **Features**

- Faster execution
- Uses index properly

- Best for numbers, IDs, exact strings

✔ **When to use**

- Login username
- ID comparison
- Exact name search

🔹 **2. LIKE Operator**

✔ **What it does**

- Used for **pattern matching**
- Works with **wildcards**

✔ **Syntax**

```
SELECT * FROM student WHERE sname LIKE 'raji';
```

✔ **Wildcards**

Wildcard	Meaning	Example
%	Any number of characters	'ra%'
_	Exactly one character	'r_ji'

✔ **Examples**

```
LIKE 'ra%'   -- starts with ra
LIKE '%ji'   -- ends with ji
LIKE '%aj%'  -- contains aj
LIKE 'r_ji'  -- raji, riji
```

✔ **Features**

- Useful for search functionality
- Slightly slower than =
- Index may not be used with % at start

✔ **When to use**

- Search box
- Filtering names

- Partial text match

🔹 3. Difference Between = and LIKE

Feature	=	LIKE
Match type	Exact	Pattern
Wildcards	❌ No	✅ Yes
Speed	Faster	Slower
Index usage	Best	Limited
Use case	Fixed values	Search & filter

🔹 4. Important Note

WHERE sname = 'raji'

WHERE sname LIKE 'raji'

👉 Both give **same result** because no wildcard is used in LIKE.

But:

- LIKE is unnecessary here
- = is better choice

✅ Conclusion (Exam Ready)

The = operator is used for exact comparison and gives faster performance, while the LIKE operator is used for pattern matching with wildcards such as % and _.
If exact value is required, = should be used.
If partial or search-based matching is needed, LIKE should be used.